Sprint 2

REQUIREMENTS

Refine the user stories that you made in the previous sprint. List your updated user stories in decreasing order of priority. Highlight the stories for which database design was completed in Sprint 1 in one color. Highlight the updated/new stories chosen for Sprint 2 in a different color. *There is no need to explicitly show your story refinement process*. Use the format shown below.

Story ID	Story description
US1	As a content manager, I want to associate recipes with specific books so that cooking activities align with literary themes.
US2	As a session facilitator, I want to track which children participate in each cooking session so that I can monitor engagement and preferences.
US3	As a child participant, I want to see which cooking sessions match my age and dietary needs so that I can participate safely and enjoyably.
US4	As a session facilitator, I want to filter recipes by allergens and ingredients so that I can plan safe sessions for children with restrictions.
US5	As a session facilitator, I want to view each child's engagement history so that I can tailor future sessions to their interests and skill levels.
US6	As a content manager, I want to identify which recipes are most popular so that I can better select books and recipes for future sessions
US7	As a content manager, I want to view summarized information in organized dashboards so that managing content is easier.
US8	As an administrator, I want session data to be automatically validated and maintained so that database consistency is preserved without manual checks.

CONCEPTUAL DESIGN

Include your complete updated conceptual design here. Use the format shown below.

Entity: **Books**
Attributes:

- ISBN (PK)
- Title
- Author
- Genre
- Age_group

Entity: **Recipes**
Attributes:

- Recipe_ID (PK)
- Recipe_name
- Instructions
- Difficulty_level
- Prep_time
- Associated_book_ISBN (FK -> Books.ISBN)

Entity: **User**
Attributes:

- ChildID (PK)
- Child_name
- Age
- Reading_level
- Dietary_restrictions

Entity: **Cooking_Sessions**
Attributes:

- Session_ID (PK)
- Session_date
- Recipe_made (FK -> Recipes.RescipeID)
- Adult_in_charge
- Rating

Entity: **Themes**
Attributes:

- Theme_ID (PK)
- Theme_Name

- Description

Entity: **Ingredients**
Attributes:
- Ingredient_ID (PK)
- Ingredient_Name
- Category
- Common_Allergens

Entity: **Sessions_Participants**
(*Associative Entity for Many-to-Many)*
Attributes:
- Session_ID(FK -> Cooking_Sessions.Session_ID)
- ChildID (FK -> User.ChildID)
- Composite PK: Session_ID + ChildID

Entity: **Book_Theme**
(Associative Entity for Many-to-Many)
Attributes:
- ISBN (FK -> Books.ISBN)
- Theme_IS (FK -> Themes.Theme_ID)
- Composite PK: ISBN + Theme_ID

Entity: **Recipe_Ingredients**
(Associative Entity for Many-to-Many)
Attributes:
- Recipe_ID (FK -> Recipes.Recipe_ID)
- Ingredient_ID (FK -> Ingredients.Ingredient_ID)
- Quantity
- Measurement
- Composite PK: Recipe_ID + Ingredient_ID

Relationship: **Books** contains **Recipes**
Cardinality: One to Many
Participation:
Book has partial participation
Recipe has total participation

Relationship: **Cooking_Sessions** uses **Recipes**
Cardinality: Many to One

Participation:
Cooking_Sessions has total participation
Recipe has partial participation

Relationship: **User** participates_in **Cooking_Session**
Implemented via Session_Participants
Cardinality: Many to Many
Participation:
User has partial participation
Cooking_Session has partial participation

Relationship: **Books** categorized_by **Themes**
Implemented via Book_Themes
Cardinality: Many to Many
Participation:
Book has partial participation
Theme has partial participation

Relationship: **Recipes** requires **Ingredients**
Implemented via Recipe_Ingredients
Cardinality: Many to Many
Participation:
Recipes has total participation
Ingredients has partial participation

LOGICAL DESIGN WITH HIGHEST NORMAL FORM IDENTIFICATION

Include your complete updated logical design here. Use the format shown below.

Entity: **Books**
Columns:

- ISBN (VARCHAR(13), PK)
- Title (VARCHAR(255))
- Author (VARCHAR(255))
- Genre (VARCHAR(100))
- Age_group (VARCHAR(255))

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Columns: Title
- Justification: Improves search/filter operations by book title.

Entity: **Recipes**

Columns:

- Recipe_ID (INT, PK)
- Recipe_name (VARCHAR(255))
- Instructions (TEXT)
- Difficulty_level (VARCHAR(50))
- Prep_time (INT)
- Associated_book_ISBN (VARCHAR(13), FK -> Books.ISBN)

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Columns: Associated_book_ISBN
- Justification: Speeds up joins between Recipes and Books.

Entity: **User**

Columns:

- ChildID (INT, PK)
- Child_name (VARCHAR(255))
- Age (INT)
- Reading_level (VARCHAR(50))
- Dietary_restrictions (VARCHAR(255))

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
  Columns: Age
  Justification: Helps identify age-appropriate sessions quickly.
- Index #2: non-clustered
  Columns: Dietary_restrictions
  Justification: Supports allergen-safe filtering for children with dietary needs.

Entity: **Cooking_Sessions**

Columns:

- Session ID (INT, PK)
- Session_date (DATE)
- Recipe made (INT, FK -> Recipes.Rescipe_ID)

- Adult_in_charge (VARCHAR(255))
- Rating (INT)

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: Recipe_made
- Justification: Improves recipe-based popularity and analytics queries.

Entity: **Themes**

Columns:

- Theme_ID (PK)
- Theme_Name (VARCHAR(255))
- Description (TEXT)

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: Theme_Name
- Justification: Enables faster searching for books or recipes by theme.

Entity: **Ingredients**

Columns:

- Ingredient_ID (INT, PK)
- Ingredient_Name (VARCHAR(255))
- Category (VARCHAR(255))
- Common_Allergens (VARCHAR(255))

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: Common_Allergens
- Justification: Supports allergen filtering

Entity: **Session_Participants**

Columns:

- Session_ID (INT, FK -> Cooking_Sessions.Session_ID)
- Child_ID (INT, FK -> User.ChildID)
- Composite PK: SessionId + Child ID

Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: ChildID

- Justification: Speeds up engagement/history queries.

Entity: **Book_Themes**
Columns:

- ISBN (VARCHAR(13), FK -> Books.ISBN)
- Theme ID (INT, FK -> Themes.Theme_ID)
- Composite PK: ISBN + Theme_ID
  Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: Theme_ID
- Justification: Useful for theme-based book grouping.

Entity: **Recipe_Ingredients**
Columns:

- Recipe_ID (INT, FK -> Recipes.Recipe_ID)
- Ingredient_ID (INT, FK -> Ingredients.Ingredient_ID)
- Quantity (DECIMAL(8,2))
- Measurement (VARCHAR(50))
- Composite PK: Recipe_ID + Ingredient_ID
  Highest normalization level: BCNF

Indexes:

- Index #1: non-clustered
- Column: Ingredient_ID
- Justification: Helps analyze allergens and ingredient usage patterns.

Normalization Verification

The database design was already normalized to BCNF in Sprint 1. Sprint 2 adds refinement and all tables continue to meet the normal form requirements.

1NF: All relations use atomic values with no repeating groups. Many-to-Many relationships are properly decomposed into separate associative tables
2NF: Only tables with composite primary keys were checked. All non-key attributes depend on the full primary key, so no partial dependencies exist.
3NF: There are no transitive dependencies. Non-key attributes depend directly on their table's primary key.
BCNF: All tables satisfy BCNF. The Sprint 2 refinements did not introduce any new functional dependencies that violate it. Every determinate in the schema is a key, and no attributes determine keys improperly.

Physical Database Design

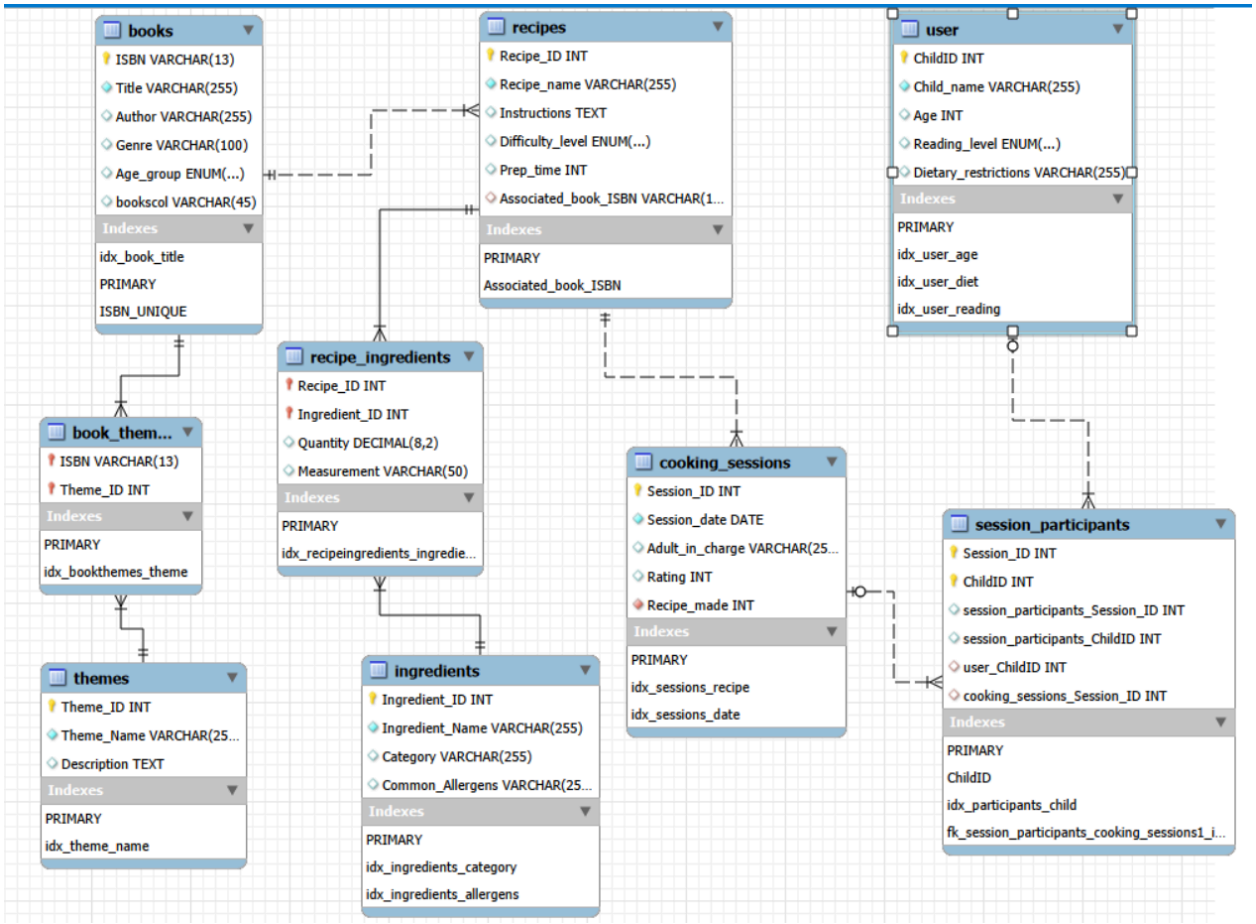
```
CREATE database IF NOT EXISTS little_chef;

USE little_chef;

CREATE TABLE IF NOT EXISTS Books(
ISBN VARCHAR(13) PRIMARY KEY, -- Changed from VARCHAR(15) for standard ISBN-13
Title VARCHAR(255) NOT NULL, -- Add NOT NULL because every book must have a title
Author VARCHAR(255),
Genre VARCHAR(100),
Age_group ENUM('Toddler' , 'Child', 'Preteen', 'Teen') -- Changed to ENUM for consistent age
categories
);

-- Index for fast search by Title
CREATE INDEX idx_books_title ON Books(Title); -- Added index for search optimization

CREATE table IF NOT EXISTS Recipes(
Recipe_ID INT primary key auto_increment, -- Added auto_increment for efficiency
Recipe_name VARCHAR(255) NOT NULL, -- Added NOT NULL because recipe name is
required
Instructions TEXT,
Difficulty_level ENUM('Easy','Medium','Hard'), -- Changed to enum to standardize difficulty
Prep_time INT,
Associated_book_ISBN VARCHAR(13),
foreign key (Associated_book_ISBN) REFERENCES Books(ISBN) ON DELETE SET NULL --
Added ON DELETE SET NULL for data inegrity
);

-- Indexes
CREATE INDEX idx_recipes_book ON Recipes(Associated_book_ISBN); -- Added index for
faster joins with Books
CREATE INDEX idx_recipes_name ON Recipes(Recipe_name); -- Added index to speed up
recipe name search


CREATE TABLE IF NOT EXISTS User (
    ChildID INT PRIMARY KEY auto_increment, -- Added auto_increment for efficiency
    Child_name VARCHAR(255) NOT NULL, -- Added NOT NULL because Child name is
required
    Age INT,
    Reading_level ENUM('Beginner','Intermediate','Advanced'), -- Changed to ENUM for
standardization
    Dietary_restrictions VARCHAR(255)
```

```
);

-- Indexes
CREATE INDEX idx_user_age ON User(Age); -- Added index for age-based filtering
CREATE INDEX idx_user_diet ON User(Dietary_restrictions); -- Added index for allergen-safe
filtering
CREATE INDEX idx_user_reading ON User(Reading_level); -- Added index for filtering by
reading level

CREATE TABLE IF NOT EXISTS Themes (
    Theme_ID INT PRIMARY KEY auto_increment, -- Added auto_increment for efficiency
    Theme_Name VARCHAR(255) NOT NULL, -- Added NOT NULL because Theme name is
required
    Description TEXT
);

-- Index
CREATE INDEX idx_theme_name ON Themes(Theme_Name); -- Added index to speed up
theme searches

CREATE TABLE IF NOT EXISTS Ingredients (
    Ingredient_ID INT PRIMARY KEY auto_increment, -- Added auto_increment for efficiency
    Ingredient_Name VARCHAR(255) NOT NULL, -- Added NOT NULL because Ingredient
name is required
    Category VARCHAR(255),
    Common_Allergens VARCHAR(255)
);

-- Indexes
CREATE INDEX idx_ingredients_allergens ON Ingredients(Common_Allergens); -- Added index
for allergen filtering
CREATE INDEX idx_ingredients_category ON Ingredients(Category); -- Added index for
ingredient category filtering

CREATE TABLE IF NOT EXISTS Cooking_Sessions (
    Session_ID INT PRIMARY KEY auto_increment, -- Added auto_increment for efficiency
    Session_date DATE NOT NULL, -- Added NOT NULL because Session Date is required
    Recipe_made INT,
    Adult_in_charge VARCHAR(255),
    Rating INT CHECK (Rating BETWEEN 1 AND 5), -- Added CHECK constraint for valid rating
values
    FOREIGN KEY (Recipe_made) REFERENCES Recipes(Recipe_ID) ON DELETE CASCADE
-- Added ON DELETE CASCADE for data integrity
);
```

```
-- Indexes
CREATE INDEX idx_sessions_recipe ON Cooking_Sessions(Recipe_made); -- Added index to speed up recipe analytics
CREATE INDEX idx_sessions_date ON Cooking_Sessions(Session_date); -- Added index for chronological queries

CREATE TABLE IF NOT EXISTS Session_Participants (
    Session_ID INT,
    ChildID INT,
    PRIMARY KEY (Session_ID, ChildID),
    FOREIGN KEY (Session_ID) REFERENCES Cooking_Sessions(Session_ID),
    FOREIGN KEY (ChildID) REFERENCES User(ChildID)
);

-- Index
CREATE INDEX idx_participants_child ON Session_Participants(ChildID); -- Added index to speed up participant history queries

CREATE TABLE IF NOT EXISTS Book_Themes (
    ISBN VARCHAR(13),
    Theme_ID INT,
    PRIMARY KEY (ISBN, Theme_ID),
    FOREIGN KEY (ISBN) REFERENCES Books(ISBN) ON DELETE CASCADE, -- Added ON DELETE CASCADE for data integrity
    FOREIGN KEY (Theme_ID) REFERENCES Themes(Theme_ID) ON DELETE CASCADE -- Added ON DELETE CASCADE for data integrity
);

-- Index
CREATE INDEX idx_bookthemes_theme ON Book_Themes(Theme_ID); -- Added index for theme-based book grouping

CREATE TABLE IF NOT EXISTS Recipe_Ingredients (
    Recipe_ID INT,
    Ingredient_ID INT,
    Quantity DECIMAL(8,2),
    Measurement VARCHAR(50),
    PRIMARY KEY (Recipe_ID, Ingredient_ID),
    FOREIGN KEY (Recipe_ID) REFERENCES Recipes(Recipe_ID)ON DELETE CASCADE, -- Added ON DELETE CASCADE for data integrity
    FOREIGN KEY (Ingredient_ID) REFERENCES Ingredients(Ingredient_ID) ON DELETE CASCADE -- Added ON DELETE CASCADE for data integrity
);
```

-- Index
CREATE INDEX idx_recipeingredients_ingredient ON Recipe_Ingredients(Ingredient_ID); -- Added index for ingredient usage and allergen filtering

ER Model



SQL QUERIES

Refine your SQL queries that you designed in the previous sprint if in need. List at least **three complex** SQL queries that perform data retrievals relevant to the features chosen in the current sprint. For each query, paste a **screenshot** of the output, as shown through your user interface.

Query 1

Description: Finds age-appropriate recipes while filtering out common allergens. Helps children find safe cooking sessions and assists facilitators in planning allergen safe activities.

Code:

```
use little_chef;

SELECT
    r.Recipe_ID,
    r.Recipe_name,
    r.Difficulty_level,
    r.Prep_time,
    b.Title AS Book_Title,
    b.Age_group,
    GROUP_CONCAT(DISTINCT i.Ingredient_Name) AS Ingredients,
    GROUP_CONCAT(DISTINCT i.Common_Allergens) AS Potential_Allergens
FROM Recipes r
JOIN Books b ON r.Associated_book_ISBN = b.ISBN
JOIN Recipe_Ingredients ri ON r.Recipe_ID = ri.Recipe_ID
JOIN Ingredients i ON ri.Ingredient_ID = i.Ingredient_ID
WHERE b.Age_group = 'Child'  -- Example age group
  AND r.Recipe_ID NOT IN (
    SELECT DISTINCT ri2.Recipe_ID
    FROM Recipe_Ingredients ri2
    JOIN Ingredients i2 ON ri2.Ingredient_ID = i2.Ingredient_ID
        WHERE i2.Common_Allergens LIKE '%nuts%'  -- Example allergen restriction
)
GROUP BY r.Recipe_ID, r.Recipe_name, r.Difficulty_level, r.Prep_time, b.Title, b.Age_group
HAVING Potential_Allergens NOT LIKE '%dairy%'  -- Additional allergen filter
ORDER BY r.Difficulty_level, r.Prep_time;
```

Output:

Recipe_ID	Recipe_name	Difficulty_level	Prep_time	Book_Title	Age_group	Ingredients	Potential_Allergens
1	Caterpillar Fruit Salad	Easy	15	The Very Hungry Caterpillar	Child	Apple,Banana,Strawberry	None
8	Mouse Cookie Snacks	Easy	25	If You Give a Mouse a Cookie	Child	Eggs,Flour,Sugar	Eggs,Gluten,None
7	Stone Soup Vegetable Stew	Easy	40	Stone Soup	Child	Apple Juice,Cinnamon,Pumpkin Puree	None
5	Green Eggs Scramble	Medium	20	Green Eggs and Ham	Child	Eggs,Ham,Spinach	Eggs,None

Query 2

Description: Analyzes individual child participation history, preferences, and session ratings. Helps facilitators tailor future sessions to children's interests and skill levels.

Code:

```
use little_chef;

SELECT
    u.ChildID,
    u.Child_name,
    u.Age,
    u.Reading_level,
    COUNT(DISTINCT sp.Session_ID) AS Sessions_Attended,
    COUNT(DISTINCT cs.Recipe_made) AS Unique_Recipes_Tried,
    AVG(cs.Rating) AS Average_Session_Rating,
    GROUP_CONCAT(DISTINCT b.Genre) AS Book_Genres_Experienced,
    GROUP_CONCAT(DISTINCT r.Difficulty_level) AS Difficulty_Levels_Tried,
    -- Most recent session details
    (SELECT r2.Recipe_name
     FROM Cooking_Sessions cs2
     JOIN Recipes r2 ON cs2.Recipe_made = r2.Recipe_ID
     JOIN Session_Participants sp2 ON cs2.Session_ID = sp2.Session_ID
     WHERE sp2.ChildID = u.ChildID
     ORDER BY cs2.Session_date DESC
     LIMIT 1) AS Most_Recent_Recipe
FROM User u
LEFT JOIN Session_Participants sp ON u.ChildID = sp.ChildID
LEFT JOIN Cooking_Sessions cs ON sp.Session_ID = cs.Session_ID
LEFT JOIN Recipes r ON cs.Recipe_made = r.Recipe_ID
LEFT JOIN Books b ON r.Associated_book_ISBN = b.ISBN
GROUP BY u.ChildID, u.Child_name, u.Age, u.Reading_level
```

HAVING Sessions_Attended > 0 -- Only show children who attended sessions
ORDER BY Sessions_Attended DESC, Average_Session_Rating DESC;

Output:

ChildID	Child_name	Age	Reading_level	Sessions_Attended	Unique_Recipes_Tried	Average_Session_Rating	Book_Genres_Experienced	Difficulty_Levels_Tried	Most_Recent_Recipe
3	Sophia Martinez	8	Intermediate	7	5	4.7143	Fantasy,Picture Book	Easy,Medium	Magic Wand Cookies
5	Olivia Brown	7	Beginner	7	5	4.1429	Fantasy,Picture Book	Easy,Medium	Magic Wand Cookies
9	Ethan Wilson	8	Intermediate	6	5	4.6667	Fantasy,Picture Book	Easy,Medium	Magic Wand Cookies
4	Noah Williams	5	Pre-reader	6	5	4.3333	Fantasy,Folktale,Picture Book	Easy,Medium	Charlotte's Web Pancakes
2	Liam Chen	6	Beginner	6	5	4.1667	Fantasy,Picture Book	Easy,Medium	Charlotte's Web Pancakes
7	Mason Lee	9	Intermediate	6	5	4.1667	Fantasy,Folktale,Picture Book	Easy	Magic Wand Cookies
6	Ava Garcia	6	Beginner	5	4	4.8000	Fantasy,Picture Book	Easy,Medium	Charlotte's Web Pancakes
1	Emma Johnson	7	Beginner	5	4	4.2000	Fantasy,Folktale,Picture Book	Easy	Caterpillar Fruit Salad
10	Mia Anderson	6	Beginner	5	4	4.2000	Fantasy,Folktale,Picture Book	Easy,Medium	Charlotte's Web Pancakes
8	Isabella Taylor	7	Beginner	5	4	4.0000	Fantasy,Picture Book	Easy,Medium	Charlotte's Web Pancakes
11	James Thompson	10	Advanced	5	4	4.0000	Fantasy,Picture Book	Easy,Medium	Magic Wand Cookies
12	Charlotte Davis	7	Beginner	3	3	5.0000	Fantasy,Picture Book	Easy,Medium	Magic Potion Smoothies
15	Lucas Harris	7	Beginner	3	3	5.0000	Fantasy,Picture Book	Easy,Medium	Magic Potion Smoothies
13	Benjamin Clark	8	Intermediate	3	3	4.0000	Fantasy,Folktale,Picture Book	Easy	Chamber of Secrets Pum...
14	Amelia White	9	Intermediate	3	3	3.6667	Fantasy,Picture Book	Easy,Medium	Web Weaving Pasta

Query 3

Description: Analyzes recipe popularity and thematic alignment to help content managers make data driven decisions. It calculates weighted popularity scores based on sessions usage, participant engagement, and ratings while providing insights into thematic connections between recipes and books. It also identifies participant demographics and dietary needs for each recipe, for better session planning and content selection.

Code:

```
use little_chef;

-- recipe_popularity_analytics.sql
WITH RecipeStats AS (
    SELECT
        r.Recipe_ID,
        r.Recipe_name,
        b.Title AS Associated_Book,
        b.Author,
        b.Genre,
        COUNT(DISTINCT cs.Session_ID) AS Times_Used,
        COUNT(DISTINCT sp.ChildID) AS Total_Participants,
        ROUND(AVG(cs.Rating), 2) AS Average_Rating,
        ROUND(
            (COUNT(DISTINCT cs.Session_ID) * 0.4 +
             COUNT(DISTINCT sp.ChildID) * 0.3 +
             AVG(cs.Rating) * 0.3), 2
        ) AS Popularity_Score,
```

```
        CONCAT('Ages ', MIN(u.Age), '-', MAX(u.Age)) AS Age_Range
    FROM Recipes r
    LEFT JOIN Books b ON r.Associated_book_ISBN = b.ISBN
    LEFT JOIN Cooking_Sessions cs ON r.Recipe_ID = cs.Recipe_made
    LEFT JOIN Session_Participants sp ON cs.Session_ID = sp.Session_ID
    LEFT JOIN User u ON sp.ChildID = u.ChildID
    GROUP BY r.Recipe_ID, r.Recipe_name, b.Title, b.Author, b.Genre
),
RecipeThemes AS (
    SELECT
        r.Recipe_ID,
        GROUP_CONCAT(DISTINCT t.Theme_Name ORDER BY t.Theme_Name)
AS Themes
    FROM Recipes r
    LEFT JOIN Books b ON r.Associated_book_ISBN = b.ISBN
    LEFT JOIN Book_Themes bt ON b.ISBN = bt.ISBN
    LEFT JOIN Themes t ON bt.Theme_ID = t.Theme_ID
    GROUP BY r.Recipe_ID
),
RecipeDietary AS (
    SELECT
        cs.Recipe_made AS Recipe_ID,
        GROUP_CONCAT(DISTINCT u.Dietary_restrictions) AS Dietary_Needs
    FROM Cooking_Sessions cs
    JOIN Session_Participants sp ON cs.Session_ID = sp.Session_ID
    JOIN User u ON sp.ChildID = u.ChildID
    WHERE u.Dietary_restrictions IS NOT NULL
      AND u.Dietary_restrictions != 'None'
      AND u.Dietary_restrictions != ''
    GROUP BY cs.Recipe_made
)
SELECT
    rs.Recipe_ID,
    rs.Recipe_name,
    rs.Associated_Book,
    rs.Author,
    rs.Genre,
    COALESCE(rt.Themes, 'No Themes') AS Themes,
```

```
    rs.Times_Used,
    rs.Total_Participants,
    rs.Average_Rating,
    rs.Popularity_Score,
    COALESCE(rs.Age_Range, 'No Participants') AS Age_Range,
                COALESCE(rd.Dietary_Needs, 'No Restrictions') AS
Common_Dietary_Needs
FROM RecipeStats rs
LEFT JOIN RecipeThemes rt ON rs.Recipe_ID = rt.Recipe_ID
LEFT JOIN RecipeDietary rd ON rs.Recipe_ID = rd.Recipe_ID
WHERE rs.Times_Used > 0
ORDER BY rs.Popularity_Score DESC, rs.Times_Used DESC;
```

Output:

Recipe_ID	Recipe_name	Associated_Book	Author	Genre	Themes	Times_Used	Total_Participants	Average_Rating	Popularity_Score	Age_Range	Common_Dietary_Needs
1	Caterpillar Fruit Salad	The Very Hungry Caterpillar	Eric Carle	Picture Book	Animals,Food	2	7	4.50	4.25	Ages 5-9	Dairy-free,Gluten-free,Nut allergy,Nut allergy, ...
2	Charlotte's Web Pancakes	Charlotte's Web	E.B. White	Fantasy	Animals,Friendship	2	5	4.50	3.65	Ages 5-7	Dairy-free,Egg allergy,Gluten-free,Vegetarian
3	Magic Wand Cookies	Harry Potter and the Sorcerer's Stone	J.K. Rowling	Fantasy	Adventure,Fantasy	2	5	4.50	3.65	Ages 7-10	Nut allergy,Nut allergy, Dairy-free
6	Wild Things Crown Cookies	Where the Wild Things Are	Maurice Sendak	Picture Book	Adventure,Imagination	1	5	5.00	3.40	Ages 6-8	Egg allergy,Nut allergy,Nut allergy, Dairy-free,...
9	Giving Tree Apple Pastries	The Giving Tree	Shel Silverstein	Picture Book	Friendship,Sharing	1	5	5.00	3.40	Ages 6-8	Egg allergy,Nut allergy,Nut allergy, Dairy-free,...
12	Magic Potion Smoothies	Harry Potter and the Sorcerer's Stone	J.K. Rowling	Fantasy	Adventure,Fantasy	1	5	5.00	3.40	Ages 6-8	Egg allergy,Nut allergy,Nut allergy, Dairy-free,...
4	Snowy Day Hot Cocoa	The Snowy Day	Ezra Jack Keats	Picture Book	Family,Seasons	1	5	4.00	3.10	Ages 5-9	Gluten-free
5	Green Eggs Scramble	Green Eggs and Ham	Dr. Seuss	Picture Book	Animals,Food	1	5	4.00	3.10	Ages 6-10	Dairy-free,Vegetarian
7	Stone Soup Vegetable Stew	Stone Soup	Marcia Brown	Folktale	Perseverance,Sharing	1	5	4.00	3.10	Ages 5-9	Gluten-free
10	Chamber of Secrets Pumpkin Juice	Harry Potter and the Chamber of Secrets	J.K. Rowling	Fantasy	Adventure,Fantasy	1	5	4.00	3.10	Ages 5-9	Gluten-free
11	Web Weaving Pasta	Charlotte's Web	E.B. White	Fantasy	Animals,Friendship	1	5	4.00	3.10	Ages 6-10	Dairy-free,Vegetarian
8	Mouse Cookie Snacks	If You Give a Mouse a Cookie	Laura Numeroff	Picture Book	Food,Imagination	1	5	3.00	2.80	Ages 6-10	Dairy-free,Vegetarian

VIEWS

List the views relevant to your application here. Use the format specified below.

View: safe_recipes_view
Goal: Pre-filters recipes to show only those with no known allergens, helping children and facilitators quickly find safe cooking options.

View: child_engagement_history
Goal: Provides a summary of each child's session attendance and preferences, allowing facilitators to easily tailor future activities without complex SQL.

View: recipe_popularity_dashboard
Goal: Aggregates session data, participant count, and ratings into a single dashboard view for content managers to analyze recipe popularity and thematic trends.

Design and Results Summary

For Sprint 2 I enhanced my database to support new features while maintaining data organization.

Data Integrity Improvements:

- Added NOT NULL constraints on critical fields to ensure data completeness
- Implemented ENUM types for standardized categorization replacing free text fields to prevent invalid data entires
- Add referential integrity actions including ON DELETE SET NULL for book-recipe relationships and ON DELETE CASCADE for associative tables to maintain database consistency automatically.

Performance Improvements:

- Strategic indexing on frequently searched columns and foreign keys used in joins to improve query performance.
- Added AUTO_INCREMENT to all primary keys for efficient record creation and management
- Implemented CHECK constraints on the Rating field in Cooking_Sessions to enforce valid 1-5 rating values

Implementation Results

The refined database successfully supports all Sprint 2 user stories through:

- Three complex SQL queries that demonstrate advanced data retrieval including subqueries and multiple joins.
- Strategic views that provide simplified data access patterns for different user roles.
- Comprehensive analytics for recipe popularity using weighted scoring that combines usage frequency, participant engagement, and session ratings

Lessons Learned

Throughout this project, I learned several crucial database design lessons. A strategic indexing approach proved essential for performance. Using ENUM types enforced data quality at the source, reducing application level validation. I discovered that thorough, detailed design from the start significantly reduces errors later in development. The importance of balancing normalization with practicality became clear while using techniques like GROUP_CONCAT in views for user friendly outputs while maintaining BCNF.